GRADUATION PROJECT · 2026

VisionLink

A Wearable Industrial Assistant on Raspberry Pi 4B

Master Technical Reference

Team: Alaa · Ali Salih · F-Alaa · Defne

Branch `openai-sdk` · 2026-05-08

This document explains every layer of VisionLink — the physical hardware, the software stack, the cloud services, the AI models, the database, the two web dashboards, and the end-to-end journey of every button press. It is written to be readable from cover to cover and to answer any technical question that might come up in a project review.

---

1. What VisionLink Is

VisionLink is a wearable assistant for factory workers, built like smart glasses (or a helmet attachment) and powered by a Raspberry Pi 4B. The worker has six physical buttons within reach. Each button triggers a different hands-free workflow — capturing evidence, asking the AI a question, or raising an emergency.

Everything the worker captures is sent to a cloud database where a supervisor watches it stream in live on a web dashboard.

The system has **two operating modes**:

Mode	Buttons	Purpose
Documentation Mode	B1, B2, B3	Hands-free recording of work activity — sessions, photos, videos, voice notes
AI Assistant Mode	B4, B5, B6	Voice conversation with an AI that can see, search the parts catalog, log incidents, request spare parts, and send emails

A separate **panic mode** rides on B6 — a double-click triggers an SOS that emails the safety officer, starts an emergency-trained AI session, and streams live camera frames to the supervisor until they manually shut it off.

---

2. The Hardware

VisionLink runs on consumer Raspberry Pi hardware with off-the-shelf sensors. Total parts cost is under €120.

2.1 Component List

Component	Model	Role	Connection
Computer	Raspberry Pi 4B (8 GB RAM)	The brain. Runs all software locally.	USB-C 5 V / 3 A
Camera	Pi Camera Module 3	Photos, videos, live frames for AI vision and SOS	CSI ribbon cable
Microphone	SPH0645LM4H (GY-SPH0645 board)	Voice input for the AI and voice notes	I²S digital — GPIO 18 (BCLK), 19 (LRCLK), 20 (data)
Amplifier	MAX98357A	Drives the speaker from a digital I²S signal	I²S digital — GPIO 18 (BCLK), 19 (LRCLK), 21 (data)
Speaker	8 Ω 1 W oval (Adafruit-style)	Audio output	Screw terminals on the amp
Buttons	6 × tactile push-button switches	Worker input	GPIO 17, 27, 22, 5, 6, 13 with shared GND
Power	USB-C power supply or USB power bank	Standalone power for the wearable	USB-C

2.2 GPIO Pin Map

The Pi 4B exposes a 40-pin GPIO header. VisionLink uses the following pins (BCM numbering):

Function	BCM GPIO	Physical Pin	Notes
B1 — Documentation Session	17	11	Falling edge interrupt, internal pull-up
B2 — Photo / Video	27	13	Falling edge interrupt, internal pull-up
B3 — Voice Note	22	15	Falling edge interrupt, internal pull-up
B4 — AI Voice	5	29	Falling edge interrupt, internal pull-up
B5 — AI Voice + Vision	6	31	Falling edge interrupt, internal pull-up
B6 — Agent / SOS	13	33	Falling edge interrupt, internal pull-up
Mic BCLK + Amp BCLK	18	12	Shared I²S bit clock
Mic LRCLK + Amp LRCLK	19	35	Shared I²S word-select clock
Mic Data Out	20	38	Microphone audio bits
Amp Data In	21	40	Amplifier audio bits
Mic 3 V	—	1	3.3 V power for mic board
Amp Vin	—	2	5 V power for amplifier
Common Ground	—	6, 14, 20, 34, 39	Shared ground rail

2.3 The Wiring Philosophy

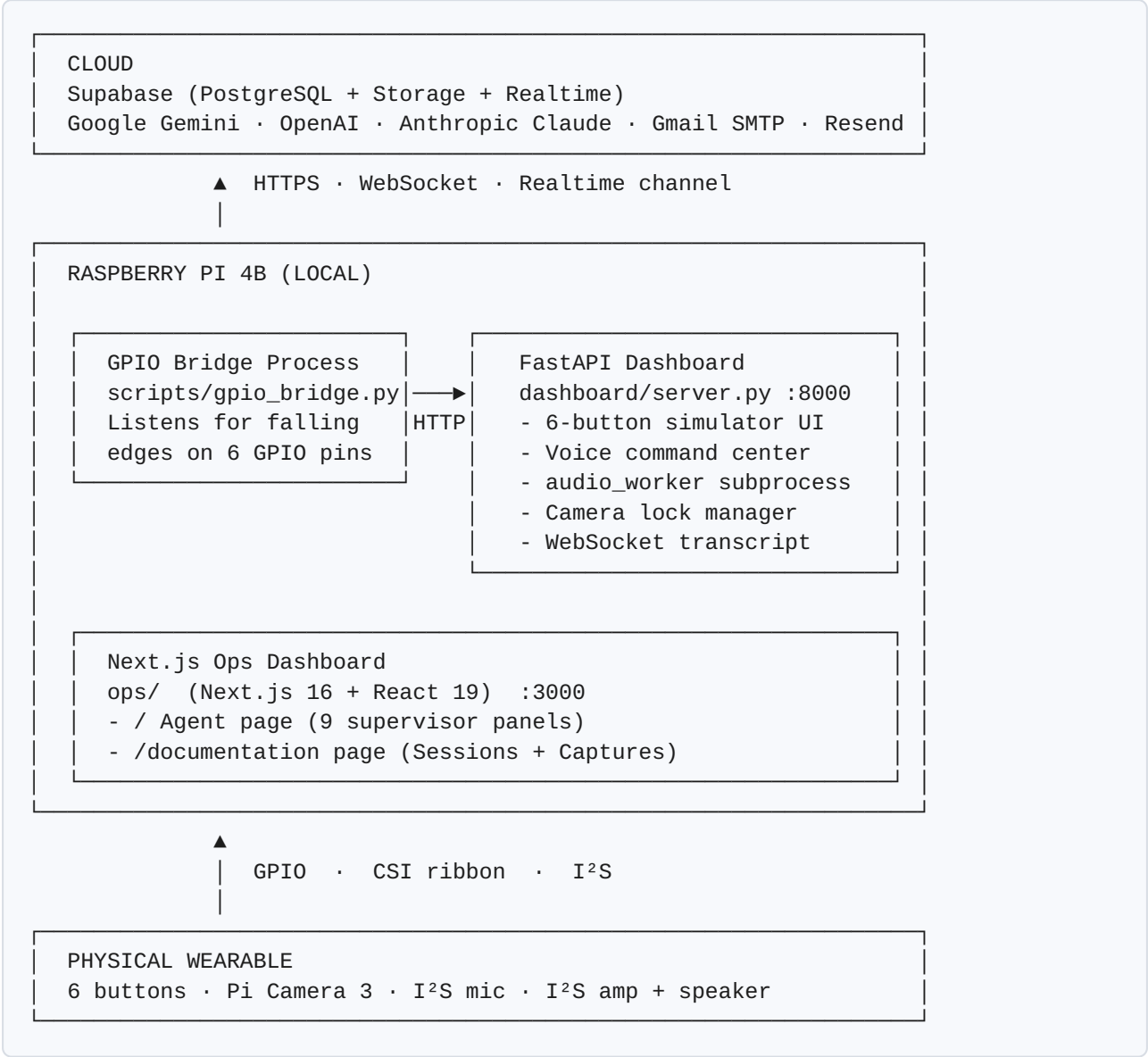
Each button is wired between its GPIO pin and ground. Pressing the button shorts the pin to ground, which the Pi's internal pull-up resistor detects as a falling edge. A short Python program listens for those edges and turns each press into a software event with debounce, double-press, and hold-to-record gestures.

The microphone and amplifier share the same clock lines (a property of the I²S bus protocol) so all four audio wires plus power and ground fit comfortably alongside the buttons on a single breadboard.

---

3. The Software Stack

VisionLink has three software surfaces and one cloud:



3.1 Languages

Language	Where it runs	Role
Python 3.13	Raspberry Pi	Backend FastAPI server, GPIO handling, AI integration, audio capture, picamera2 control, all cloud API calls
TypeScript	Browser (compiled to JS by Next.js)	Ops dashboard — 9 live data panels with full CRUD
JavaScript (vanilla)	Browser	Voice command center — the simulated wearable UI on the worker side
HTML / Tailwind CSS 4	Browser	Layout and styling for both dashboards
Bash	Raspberry Pi	System orchestration, video capture script (legacy), launch scripts
SQL	Supabase	Database schema, indexes, realtime publication, row-level security policies

3.2 Frameworks and Libraries

Layer	Technology	Why it was chosen
HTTP / WebSocket server	**FastAPI** + **Uvicorn**	Python-native, async-first, automatic OpenAPI docs, perfect for real-time event handling
Frontend (supervisor)	**Next.js 16** + **React 19**	App-router, server components, Turbopack — modern fast dev experience
Styling	**Tailwind CSS 4**	Utility-first, no separate CSS files, consistent design system
Database client (Python)	**supabase-py**	Official, supports realtime subscriptions and storage uploads
Database client (browser)	**@supabase/supabase-js**	Official, used for live subscriptions and direct CRUD
GPIO handling	**RPi.GPIO 0.7.2** + **lgpio** + **gpiozero**	Standard Pi GPIO library; falling-edge interrupts with debounce
Camera	**picamera2**	Official modern Pi Camera library with hardware MMAL acceleration
Audio capture/ playback	**sounddevice** + **arecord/aplay** + custom AudioBridge	Direct I²S device access, low-latency mic feed
Voice AI	**google-genai** SDK (Gemini Live) and **OpenAI SDK** (Realtime API)	Bidirectional streaming voice + vision
Email	**smtplib** (Gmail) and **resend** (Resend)	Two providers — Gmail is the default
QR codes	**pyzbar**	Pi Camera frame → QR string decode
Image utilities	**Pillow**, **opencv-python-headless**	Resizing, encoding, JPEG compression

4. The Six Buttons — Documentation Mode (B1, B2, B3)

The first three buttons let the worker capture evidence of a shift without ever taking off their gloves. Everything captured during a "session" is automatically grouped together in the database and shown to the supervisor in real time.

4.1 B1 — Documentation Session toggle (GPIO 17)

Single press. Opens or closes a documentation session — the container that bundles together all photos, videos, and voice notes captured during that period of work.

The journey:

1. Worker presses B1. The pin drops from 3.3 V to 0 V.
2. The Linux kernel raises a falling-edge interrupt on GPIO 17.
3. `scripts/gpio_bridge.py` (a small Python sidecar) receives the interrupt via RPi.GPIO and POSTs `/api/button/1/single` to the FastAPI dashboard on port 8000.
4. The dashboard dispatches to the `b1_doc_session_toggle` handler.
5. If no session is currently open: insert a new row into the `sessions` table in Supabase, with the current timestamp and a label like *"Session 2026-05-08 14:32"*. Status: `open`.
6. If a session is already open: update that row to `status='closed'` and `ended_at=now()`.
7. Supabase pushes the change over its realtime channel. Within ~200 ms, the supervisor's *Documentation* page reflects the new state — the open-session row glows green, and the *Sessions* count ticks up.

4.2 B2 — Photo / Video (GPIO 27)

Single press = photo. Double press = 5-second silent video.

Photo journey:

1. B2 single press → `b2_take_photo` handler.
2. picamera2 grabs a 1024×576 JPEG from the Pi Camera 3.
3. The JPEG bytes are uploaded to the Supabase Storage bucket `session-assets` under the path `<session_id>/photo_<ts>.jpg`.
4. A row is inserted into `session_assets` with `kind='photo'` and the storage path.
5. The supervisor's *Captures* panel shows the new photo as a thumbnail. Clicking it opens a full-size modal preview.

Video journey:

1. B2 double press → `b2_record_video` handler.

2. A subprocess runs `rpicam-vid --codec libav --libav-format mp4 -t 5000` which produces a 5-second 1280×720 30 fps MP4 directly.
3. The MP4 file is uploaded to the same `session-assets` bucket.
4. Row in `session_assets` with `kind='video'` and `duration_s=5`.
5. Plays inline in the supervisor's *Captures* panel via the browser's native `<video>` element.

> **Why no audio in the video.** The I²S microphone is held exclusively by `audio_worker` (the always-running subprocess that feeds AI sessions and B3 voice notes). A parallel `arecord` would fail with *device busy*. Audio mux through the existing AudioBridge is a planned improvement.

4.3 B3 — Voice Note (GPIO 22)

Hold to record. Release to upload.

Journey:

1. B3 press → `hold_start` event → `b3_voice_note_start` handler.
2. The handler subscribes to the AudioBridge mic stream and starts accumulating raw 16 kHz mono PCM bytes in memory.
3. Worker speaks (e.g. *"check valve B7 tomorrow morning"*).
4. Worker releases B3 → `hold_end` event → `b3_voice_note_end` handler.
5. The accumulated PCM is wrapped in a WAV header.
6. The WAV file is uploaded to `session-assets/<session_id>/voice_<ts>.wav`.
7. A row in `session_assets` with `kind='voice_note'` and `duration_s` computed from the press duration.
8. The supervisor's *Captures* panel shows a playable `<audio>` element with controls.

> **Guard.** B3 refuses to start while any AI session (B4, B5, or B6) is using the microphone. The user gets an explicit error rather than a corrupted recording where the audio is split between two consumers.

---

5. The Six Buttons — AI Assistant Mode (B4, B5, B6)

The last three buttons turn VisionLink into a hands-free conversational agent. Pressing one of these starts a streaming voice session with a cloud AI model. The AI hears the worker through the I²S mic and speaks back through the I²S amplifier and speaker.

5.1 B4 — AI Voice (GPIO 5)

Single press to start, double press to stop.

Journey:

1. B4 single press → `b4_ai_voice_only` handler.
2. The handler reads `wearable_settings.b4_provider` from Supabase — either `gemini` or `openai`. The supervisor controls this from the Wearable Settings panel.
3. A bidirectional WebSocket connection is opened to either Google Gemini Live API or OpenAI Realtime API.
4. The local AudioBridge starts forwarding mic audio to the model and playing model responses back through the speaker.
5. The same six tools (parts lookup, incident logging, etc.) are exposed to the model so it can take action on behalf of the worker.
6. Live transcripts stream back over a WebSocket to the voice command center and appear in real time.
7. A double-press of B4 closes the WebSocket and stops audio routing.

5.2 B5 — AI Voice + Vision (GPIO 6)

Single press to start, single press again to take a snapshot, double press to stop.

This is the most powerful button. It opens an AI session that can *see* through the Pi Camera in addition to hearing the worker.

Journey:

1. B5 single press → `b5_ai_voice_vision` handler.
2. Same as B4 but the AI session is configured with a vision channel.
3. By default the wearable uses **snap on press**: the first B5 press starts the session and triggers an automatic camera snapshot to show the AI what the worker is looking at; subsequent single presses inject fresh snapshots into the same conversation.
4. Other vision modes selectable from the Wearable Settings panel: `gemini_video` (continuous 1 fps stream — Gemini-only) and `auto_snap_4s` (auto-snap every 4 seconds — both providers).

5. Worker can ask things like *"what's wrong with this gauge?"* or *"what's this part code?"* — the AI sees the image and responds.
6. Behind the scenes: each snapshot is a small JPEG sent to the model over the same WebSocket as the audio stream.

5.3 B6 — Agent / SOS Panic Mode (GPIO 13)

B6 has two distinct behaviors:

- **Single press:** the worker is warned via a toast message that SOS requires a double-click — pocket-dial protection.
- **Double press:** a full SOS panic mode is triggered.

SOS journey (the most complex flow in the entire system):

1. B6 double press → `b6_sos_trigger` handler.
2. A new row is inserted into `sos_events` with worker identity and `triggered_at = now()`. The row's UUID becomes the SOS ID.
3. **Email the safety officer.** A fire-and-forget task uses the `send_report` tool internally to look up the safety officer's email from the `managers` table and send an alert.
4. **Auto-close any open documentation session.** Worker shouldn't be doing paperwork in an emergency, so the open session is updated to `status='closed'` automatically.
5. **Start an OpenAI Realtime AI session** with a calm-emergency system prompt — *"Stay calm, ask short questions, describe what you see, tell them help is on the way. Do NOT call any tools right now."* The voice "Cedar" with reasoning_effort "medium".
6. **Auto-snap a fresh camera frame every N seconds** (default 10 seconds, tunable). Each frame is uploaded to `sos/<sos_id>/frame_<ts>.jpg`, then injected into the OpenAI session via `send_image()`. The `frames_sent` counter on the `sos_events` row increments.
7. **Stream the live transcript** of what the AI says into `sos_events.live_transcript`. The supervisor watches the stream appear character-by-character on their dashboard.
8. **The supervisor's SOS panel goes red and starts pulsing** with a two-tone audible alarm (Web Audio API — no MP3 file needed).
9. **The supervisor can shut it off.** A big red SHUT OFF button opens a modal asking for the supervisor's name and reason. On confirm, the row is updated to `resolved=true` with the supervisor name. The wearable is polling the row every 2 seconds — within ~2 s, the AI session is closed, the auto-snap loop stops, and the wearable falls silent.
10. **Hard timeout.** If the supervisor never sees the alert, a server-side `asyncio.wait_for` auto-resolves the SOS after the configured `sos_max_duration_s` (default 600 s = 10 minutes) to bound the audio session length and storage cost.
11. The worker can also cancel the SOS themselves with another B6 double-click — the running task's resolution watcher sees the flipped flag and tears everything down.

---

6. The AI Brains

VisionLink uses three different cloud AI providers, each chosen for a specific strength.

6.1 Google Gemini Live (default)

Provider: Google AI Studio **Model:** `gemini-2.5-flash` (and `gemini-3.1-flash-live-preview` for streaming) **Where used:** B4 voice sessions, B5 vision sessions, default for both buttons unless overridden in Wearable Settings.

Why: Multimodal native (audio + image + text in the same stream), fastest first-token latency among the three providers, lowest cost per minute. Excellent at general conversation and tool calling.

6.2 OpenAI GPT Realtime 2

Provider: OpenAI **Model:** `gpt-realtime-2` (released 2026-05-07) **Where used:** Selectable alternative for B4 and B5; **always used for B6 SOS** regardless of B4/B5 settings.

Why: OpenAI's instruction-following is more reliable for tightly scripted personas like *calm emergency operator*. The voice model has the best perceived "presence" for a high-stakes situation. Speed parameter set to 1.2× by default to combat the model's natural slow pacing.

6.3 Anthropic Claude (Sonnet/Opus 4.x)

Provider: Anthropic **Model:** Claude Sonnet 4.6 (planned for email drafting) **Where used:** Currently a research target for the B6 *agent command* path — *"send a report saying X happened"* would route the voice transcript to Claude, which writes a polished email body, and Resend or Gmail SMTP delivers it.

Why: Best long-form writing quality among the three. Better at maintaining a professional tone. The plan is to use Claude for the drafting step *only* — Gemini still handles the voice conversation.

6.4 Soniox (planned alternative STT)

Provider: Soniox **Where used:** A fallback speech-to-text path if Gemini Live's built-in transcription is insufficient. Currently a stub in `src/ai/stt.py`.

Why: Industry-standard standalone STT with streaming support. Could be useful for offline transcripts or for quality verification of what Gemini "heard."

---

7. The Tools the AI Can Call

Both Gemini and OpenAI sessions expose the same six tools. Calling them is how the AI takes real action — looking up data in the database, writing rows, sending emails. Without tools the AI could only talk; with them, it can *do*.

#	Tool	What it does	Touches database
1	`lookup_component(query)`	Searches the `components` parts catalog by part code (exact) then name (fuzzy). Returns up to 3 rows with torque spec, maintenance interval, safety notes.	Read
2	`log_incident(description, category?, severity?, location?)`	Inserts a row into `incidents` whenever the worker reports something wrong. Categories: `safety`, `equipment`, `leak`, `damage`, `other`. Severities: `low`, `medium`, `high`, `critical`.	Write
3	`mark_task_complete(task_query)`	Fuzzy-matches the worker's spoken description against open rows in `worker_tasks` and marks the best match `complete`.	Write
4	`get_my_assignments(include_complete?)`	Lists the worker's pending or all tasks. Used when the worker says *"what's on my list today?"*	Read
5	`request_part(part_query, quantity?, urgency?, reason?)`	Inserts a row into `part_requests` so procurement sees that the worker needs spare parts.	Write
6	`send_report(report_name, recipient_role?, recipient_name?, recipient_email?, custom_message?)`	The killer tool. Looks up a template by name from `report_templates`, looks up a recipient by role from `managers`, auto-injects live data from `incidents` / `worker_tasks` / `part_requests`, sends via Gmail SMTP (default) or Resend, and logs the result to `sent_reports`.	Read + Write + Email

Three absolute rules in the system prompt prevent the most common failure modes:

1. **Never lie about completing actions.** If the AI says "I logged it", it MUST have already called the matching tool.

2. **Verb triggers.** Whenever the worker says a verb like *"log", "mark", "send", "request"* — the AI must call the matching tool immediately, not just acknowledge.
3. **No info-collection trap.** The AI never asks more than one clarifying question — it commits to an action with reasonable defaults instead of stalling.

---

8. The Database

VisionLink uses **Supabase** as its single backend service. Supabase is PostgreSQL with batteries included — managed hosting, file storage, realtime push, authentication, and SDKs for every popular language.

8.1 Why Supabase

Need	What Supabase provides
Persistent SQL database	Managed PostgreSQL 16
File storage (photos, videos, voice notes, SOS frames)	S3-compatible buckets with public URL generation
Live updates from DB to dashboard (no polling)	Realtime via logical replication → WebSocket
Two-language clients (Python on Pi, JS in browser)	Official `supabase-py` and `@supabase/supabase-js`
Free tier sufficient for the demo	500 MB DB, 1 GB storage, 200 MB egress/month
Easy schema management	SQL editor in the web dashboard, plus CLI

8.2 Tables

Table	Rows	Purpose
`components`	~8 seeded	Parts catalog. Read by `lookup_component`.
`worker_tasks`	growing	Tasks assigned to a worker. Read by `get_my_assignments`, written by `mark_task_complete`.
`incidents`	growing	Reported safety / equipment / leak / damage events. Written by `log_incident`.
`part_requests`	growing	Procurement queue. Written by `request_part`.
`managers`	~4 seeded	Recipient list for emails (role → email mapping). Read by `send_report`.
`report_templates`	~5 seeded	HTML email templates with placeholders. Read by `send_report`.
`sent_reports`	growing	Audit log of every email sent (success or failure, provider used). Written by `send_report`.
`sessions`	growing	B1 documentation sessions.
`session_assets`	growing	B2 photos + B2 videos + B3 voice notes. References `sessions`.
`sos_events`	growing	B6 SOS panic events, including the live transcript field.
`wearable_settings`	1 row (singleton)	Remote config — which AI provider per button, vision mode, SOS tunables, worker identity.

8.3 Storage Buckets

Bucket	Visibility	Contents
`session-assets`	Public (URLs go in DB)	All B2 photos, B2 videos, B3 voice notes, B6 SOS frames

8.4 Realtime Channel

Every table is added to the `supabase_realtime` publication. The ops dashboard subscribes once per table and receives `INSERT`, `UPDATE`, `DELETE` events over WebSocket. New rows appear in the supervisor's UI within ~200 ms of being written.

8.5 Row-Level Security

Disabled for all tables in this demo build. Production deployment would re-enable RLS with policies like *"a worker can only insert rows where worker_id = auth.uid()"*. The schema files include the `alter table ... disable row level security` calls for transparency.

---

9. The Two Web Dashboards

VisionLink has two separate web surfaces with different audiences.

9.1 Voice Command Center (worker simulator)

URL: `http://192.168.0.31:8000` (the Pi's local IP) **Tech:** FastAPI server + vanilla HTML/JS/Tailwind **Audience:** Developer or worker testing the wearable

Looks and behaves like the wearable would, but on screen. Six on-screen buttons match the six physical buttons. The same backend handlers are called whether a press comes from the screen or the GPIO bridge. Live transcripts of the AI conversation stream in the right side; recent captures show on the left. Useful for debugging.

9.2 Ops Dashboard (supervisor)

URL: `http://192.168.0.31:3000` **Tech:** Next.js 16 + React 19 + Tailwind 4 **Audience:** Factory supervisor

Two pages, switchable from the top nav:

Page 1 — Agent (`/`): the existing command center with nine realtime panels:

- Incidents · Tasks · Parts · Components
- Managers · Report Templates · Sent Reports
- SOS · Wearable Settings

Page 2 — Documentation (`/documentation`): new page showing what the worker has been capturing.

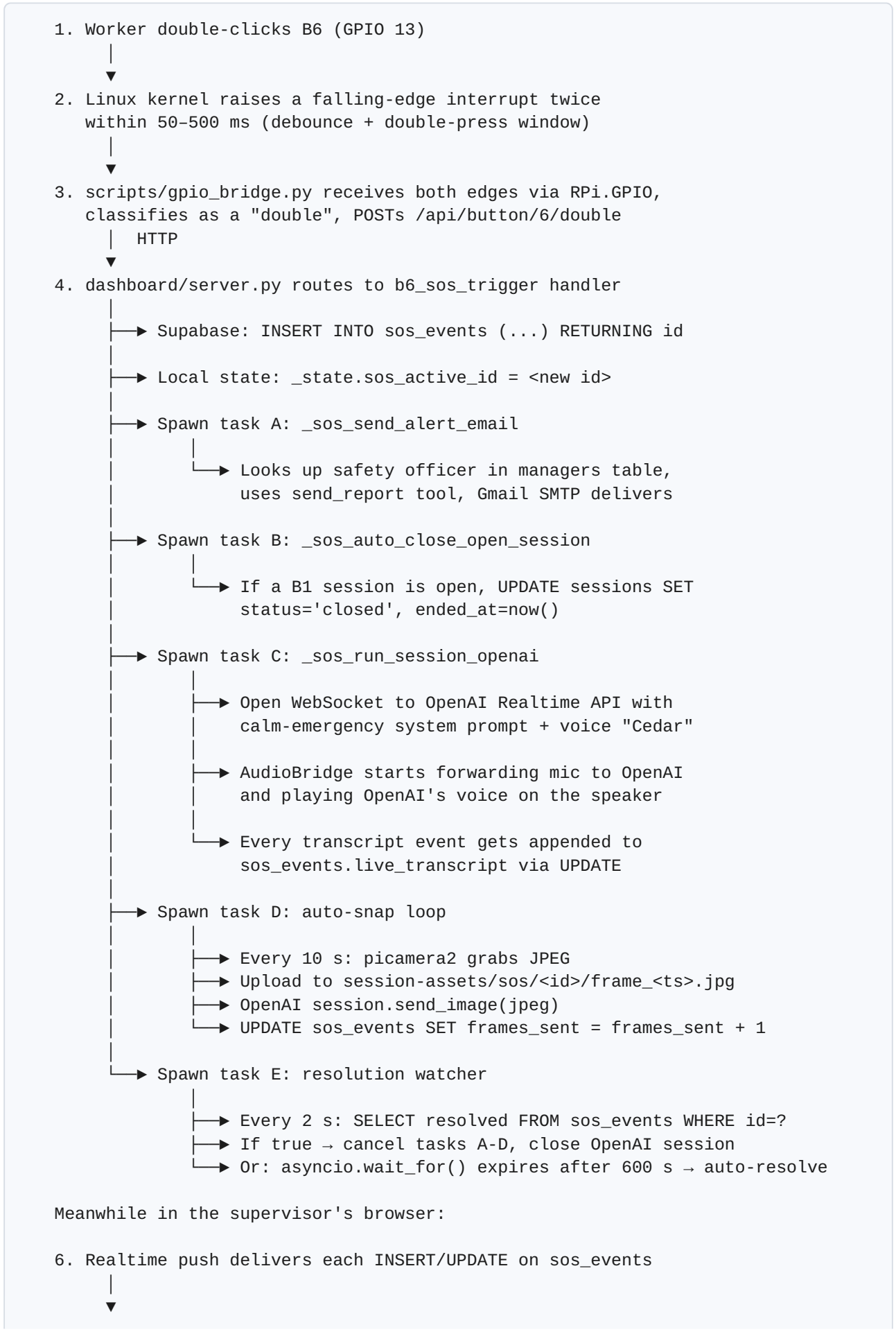
- A stats strip (sessions / open / photos / videos / voice notes / orphans)
- *Sessions* panel — list of shifts with start/end time, duration, and per-kind asset counts. Click a row to expand and see every capture in that session.
- *Captures* panel — tabbed grid (All · Photos · Videos · Voice notes · Orphans). Photos open in a modal preview. Videos use `<video>` controls. Voice notes use `<audio>` controls.

Every panel updates in real time as the worker captures things — within ~200 ms of a button press.

---

10. End-to-End Journey: One SOS Press

This section traces the full path of a single B6 double-click, from finger to email to live frames on the supervisor's screen — every component the signal touches.


1. Worker double-clicks B6 (GPIO 13)
2. Linux kernel raises a falling-edge interrupt twice
within 50–500 ms (debounce + double-press window)
3. scripts/gpio_bridge.py receives both edges via RPi.GPIO,
classifies as a "double", POSTs /api/button/6/double
HTTP
4. dashboard/server.py routes to b6_sos_trigger handler
Supabase: INSERT INTO sos_events (...) RETURNING id
Local state: _state.sos_active_id = <new id>
Spawn task A: _sos_send_alert_email
Looks up safety officer in managers table,
uses send_report tool, Gmail SMTP delivers
Spawn task B: _sos_auto_close_open_session
If a B1 session is open, UPDATE sessions SET
status='closed', ended_at=now()
Spawn task C: _sos_run_session_openai
Open WebSocket to OpenAI Realtime API with
calm-emergency system prompt + voice "Cedar"
AudioBridge starts forwarding mic to OpenAI
and playing OpenAI's voice on the speaker
Every transcript event gets appended to
sos_events.live_transcript via UPDATE
Spawn task D: auto-snap loop
Every 10 s: picamera2 grabs JPEG
Upload to session-assets/sos/<id>/frame_<ts>.jpg
OpenAI session.send_image(jpeg)
UPDATE sos_events SET frames_sent = frames_sent + 1
Spawn task E: resolution watcher
Every 2 s: SELECT resolved FROM sos_events WHERE id=?
If true → cancel tasks A-D, close OpenAI session
Or: asyncio.wait_for() expires after 600 s → auto-resolve
Meanwhile in the supervisor's browser:
6. Realtime push delivers each INSERT/UPDATE on sos_events

```
 7. SosPanel re-renders the row:
      - Border + glow turn red
      - Live transcript appears character-by-character
      - Frame thumbnail updates every 10 s with the latest snapshot
      - Two-tone audible alarm starts (Web Audio API)
      - Big SHUT OFF button is now visible
      │
      ▼
 8. Supervisor clicks SHUT OFF, types name + reason, confirms
      │
      ▼
 9. Browser sends UPDATE sos_events SET resolved=true, resolved_by=...
      │
      ▼
10. Wearable's resolution watcher sees the flag flip on its next
    2-second poll, cancels tasks A-D, closes OpenAI session,
    stops audio routing, releases the camera lock.
      │
      ▼
11. Worker hears the AI voice stop. Wearable returns to idle.
    sos_events row is now resolved=true with full audit trail.
```

This entire journey takes about 4 seconds from button press to supervisor's red alarm. The hard timeout, supervisor shutoff, and worker self-cancel all converge on the same teardown path so there's exactly one place where state is cleaned up.

---

11. Hosting and Cloud Services

VisionLink mixes locally-hosted and cloud-hosted components.

11.1 What Runs on the Pi (Local)

Process	Port	Purpose	Auto-start
FastAPI dashboard (`uvicorn dashboard.server:app`)	8000	Backend + voice command center + websocket transcript	systemd unit `visionlink.service` (planned)
Next.js dev server (`next dev`)	3000	Ops supervisor dashboard	Manual (production would use `next build` + `next start`)
GPIO bridge (`scripts/gpio_bridge.py`)	—	Translates physical button presses to HTTP calls	Manual (planned: another systemd unit)
audio_worker (sub-process of FastAPI)	—	Reads I²S mic, plays I²S speaker, owns the AudioBridge	Spawned by FastAPI on startup

The Pi is on the same Wi-Fi as the supervisor's laptop. No NAT or port forwarding is needed for the demo.

11.2 What Runs in the Cloud (External)

Service	Provider	Purpose	Auth
Supabase project	Supabase (managed)	PostgreSQL DB, file storage, realtime channel	Two API keys: `SUPABASE_URL` (public) + `SUPABASE_SERVICE_KEY` (server-only) + `SUPABASE_ANON_KEY` (browser)
Gemini Live API	Google AI Studio	Voice + vision AI sessions	`GEMINI_API_KEY`
OpenAI Realtime API	OpenAI	Voice + vision AI sessions, SOS persona	`OPENAI_API_KEY`
Anthropic API (planned)	Anthropic	Claude email drafting	`ANTHROPIC_API_KEY`
Gmail SMTP	Google	Email delivery	`SMTP_EMAIL` + `SMTP_APP_PASSWORD`
Resend (alternative)	Resend	Email delivery (developer-friendly alternative)	`RESEND_API_KEY`
Soniox (planned alternative)	Soniox	Standalone STT	`SONIOX_API_KEY`

All API keys are stored in a single `.env` file at the project root. The file is in `.gitignore` and never committed. A `.env.example` template ships with the project.

---

12. Security and Privacy Notes

Topic	Current state	Production-readiness
API keys	In a local `.env` file, gitignored	Demo-ready. Production would use a secrets manager.
Supabase Row-Level Security	Disabled for the demo	Production must enable RLS with auth-based policies
Storage bucket	`session-assets` is public	Production should use signed URLs
Email delivery	Gmail SMTP via app password	Demo-ready. App password is per-account, revocable.
Worker identity	Hardcoded to a single demo worker	Production: Supabase Auth login per worker
Local dashboard auth	None — anyone on the Wi-Fi can hit :3000 and :8000	Production: VPN, basic auth, or Supabase Auth gating
Audio recording disclosure	Voice notes stored as WAV files	Production: clear worker consent + retention policy

13. Common Project Review Questions

Q. What database does VisionLink use?

PostgreSQL, hosted via Supabase. PostgreSQL 16 with the `supabase_realtime` extension for live websocket push.

Q. How does the wearable talk to the AI?

Bidirectional WebSocket. The Pi opens a connection to either Google's Gemini Live API or OpenAI's Realtime API. Microphone audio is streamed up; the AI's voice and tool calls stream down. Tool calls are intercepted on the Pi side, executed against Supabase or Gmail, and the result is sent back into the same WebSocket.

Q. Why two AI providers?

Each has different strengths. Gemini is the cheaper, faster, multimodal-native default. OpenAI Realtime is used for the SOS persona because its instruction-following is more reliable for tightly-scripted scenarios. The supervisor can switch the default provider for B4 and B5 from the Wearable Settings panel without restarting anything.

Q. What language is the backend?

Python 3.13. The FastAPI framework handles HTTP and WebSocket endpoints. AsyncIO drives concurrent tasks like the SOS panic loop.

Q. What language is the supervisor dashboard?

TypeScript on top of Next.js 16 (App Router) with React 19 and Tailwind CSS 4. Live updates come from Supabase Realtime over WebSocket.

Q. How do photos and videos get to the supervisor?

The Pi uploads them directly to Supabase Storage (an S3-compatible bucket). A row in `session_assets` points to the storage path. The supervisor's browser subscribes to the `session_assets` table over realtime; new rows trigger a UI update; the browser fetches the public URL of the file and displays it inline.

Q. What happens if the network is down?

The wearable degrades gracefully:

- Button presses still register locally.
- B1 session opens fail (Supabase insert errors) and the worker hears an error tone.
- B2 captures buffer locally for retry, but the current implementation does not yet persist them across power cycles — this is an acknowledged improvement target.
- B4/B5/B6 AI sessions fail to connect; the worker hears a tone.
- The supervisor dashboard shows a stale view until reconnection.

Q. How is the SOS panic mode different from a normal AI session?

Five things:

1. **Always uses OpenAI** regardless of B4/B5 settings.

2. **Different system prompt** — calm-emergency persona, no tool calls allowed.
3. **Inserts a database row** at trigger time so the supervisor's dashboard can light up.
4. **Auto-snaps a fresh camera frame every N seconds** and uploads each to storage so the supervisor sees what's happening.
5. **Has a remote shutoff** — the supervisor flips `resolved=true` in the database; the wearable polls every 2 s and tears everything down.

Q. What's the GPIO bridge?

A 100-line Python sidecar (`scripts/gpio_bridge.py`) that listens for falling edges on the six button pins and translates each gesture (single, double, hold) into the same HTTP call the on-screen simulator makes. It runs as a separate process from the dashboard so that a software change in one cannot break the other. The bridge uses RPi.GPIO with a 50 ms debounce and a 500 ms double-press window.

Q. How are double-clicks detected?

Two falling edges within 500 ms of each other on the same pin. The first edge starts a timer; if a second edge arrives before the timer expires, a *double* event fires; otherwise the timer fires a *single* event. Hardware-level bouncing is rejected by RPi.GPIO's `bouncetime` parameter (50 ms).

Q. What's `audio_worker`?

A subprocess of the FastAPI server that owns the I²S audio device exclusively. It feeds the microphone bytes into a shared in-process queue (the `AudioBridge`) which both AI sessions and B3 voice notes read from, and it consumes a queue of speaker bytes from anyone that wants to play sound. Centralizing audio access this way avoids the *device busy* error you'd get from two processes opening ALSA simultaneously.

Q. What's the I²S bus?

A digital audio standard. Three wires per direction: a bit clock (BCLK), a word-select clock (LRCLK), and a data line. The microphone and amplifier share the two clock lines and have separate data lines, so the whole audio path is just four wires plus power and ground. I²S avoids the noise and latency of analog audio on the Pi.

Q. How much does this cost to run?

Per a single demo session of ~10 minutes of voice + vision:

- Gemini: ~€0.15
- OpenAI: ~€0.40
- Supabase + storage + email: free tier, no charge

A full SOS event (10-minute hard cap, ~60 frames, full audio session) costs about €0.30 on OpenAI worst case.

Q. Is the source code on GitHub?

Yes — the project is at `https://github.com/alaamjaish/visionlink` on the `openai-sdk` branch. The repository has a comprehensive `CLAUDE.md` master context file at the root and per-session logs under `Documentation/sessions/`.

---

14. Glossary

Term	What it means
BCM	Broadcom GPIO numbering (vs physical pin numbering). VisionLink's pin map uses BCM.
GPIO	General-Purpose Input/Output. The Pi's programmable pins.
Falling edge	A signal transition from high (3.3 V) to low (0 V). VisionLink's button presses are detected as falling edges.
Pull-up resistor	An internal resistor that holds an input pin at 3.3 V when nothing is connected. The Pi has these built into every GPIO.
I²S	Inter-IC Sound — a digital audio bus standard.
CSI	Camera Serial Interface — the ribbon cable port on the Pi that connects to the Pi Camera.
WebSocket	A persistent two-way connection between browser and server. Used for live transcripts and AI streaming.
Realtime	Supabase's WebSocket-based push of database changes. The dashboard receives `INSERT` / `UPDATE` / `DELETE` events as they happen.
Tool call	When the AI invokes a function (like `lookup_component`) that runs server-side and returns a result the AI can speak.
VAD	Voice Activity Detection — the model's heuristic for deciding when the worker has finished speaking.
TTS	Text-to-Speech.
STT	Speech-to-Text.
RLS	Row-Level Security — a PostgreSQL feature for per-row permissions. Disabled in this demo.
systemd	The Linux service manager. VisionLink's auto-start unit lives at `visionlink.service`.
Supabase storage bucket	An S3-compatible cloud file store. VisionLink uses one bucket: `session-assets`.

End of master reference.